МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №4
з дисципліни «Професійний Python»

Тема: «Професійне ООП та система типів Python»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Сформувати навички побудови типізованої об'єктно-орієнтованої моделі предметної області: класи, композиція і наслідування, ABC та Protocol, dataclass і value objects, properties, dunder methods, принципи SOLID, dependency injection, generic classes, TypeVar, TypedDict і статична перевірка коду через mypy.

1. Предметна область і постановка задачі

Варіант 1. Система обліку студентів (продовження ЛР1–ЛР3). Студент навчається в групі й має академічний запис – набір результатів із предметів; кожен результат складається з оцінки (0–100) і кількості кредитів. Студенти бувають бюджетні й контрактні, а стипендію отримують лише бюджетні з достатнім середнім балом. Оцінки зберігаються як `Decimal`, бо float дає похибки округлення. Спосіб обчислення середнього бала змінюється стратегією: простий, зважений за кредитами або середнє з кількох найкращих оцінок.

Таблиця 1 – Вимоги варіанта та їх реалізація

Вимога	**Реалізація**
додавання та редагування даних студента	`StudentService.register`, `edit`; `Student.edit` перевіряє нові значення до зміни стану
пошук за прізвищем/ID і групою	`find_by_last_name` (без урахування регістру), `find_by_id`, `find_in_group`
обчислення середнього бала	`AcademicRecord.average(policy)` і стратегії `GradingPolicy`
визначення найкращого студента	`StudentRegistry.best_student(policy, group)`, `Group.best`
repository	`Repository[K, T]` (ABC) і `InMemoryRepository[K, T]`
reporting service	`ReportService`: summaries, ranking, group_report, card
notification abstraction	`Notifier` (runtime_checkable Protocol) і адаптери `ConsoleNotifier`, `MemoryNotifier`

Таблиця 2 – Domain objects та інші класи моделі

Клас	**Вид**	**Відповідальність**
`Student`, `BudgetStudent`, `ContractStudent`	dataclass, entity; підкласи	дані студента, ідентичність за id, стипендія
`Group`	entity, колекція	склад групи, середній бал, найкращий
`AcademicRecord`	custom collection	результати студента (один на предмет)
`Grade`, `GroupCode`, `CourseResult`	frozen value objects	оцінка 0–100; код групи PI-21; оцінка з предмета й кредитами
`StudentRegistry`	aggregate root	інваріанти: студент завжди в існуючій групі
`GradingPolicy` і 3 підкласи	ABC, strategy	обчислення середнього бала
`StudentService`, `ReportService`	application services	сценарії та сповіщення; звітність
`StudentError` і 6 підкласів	ієрархія винятків	доменні помилки

2. Діаграма класів

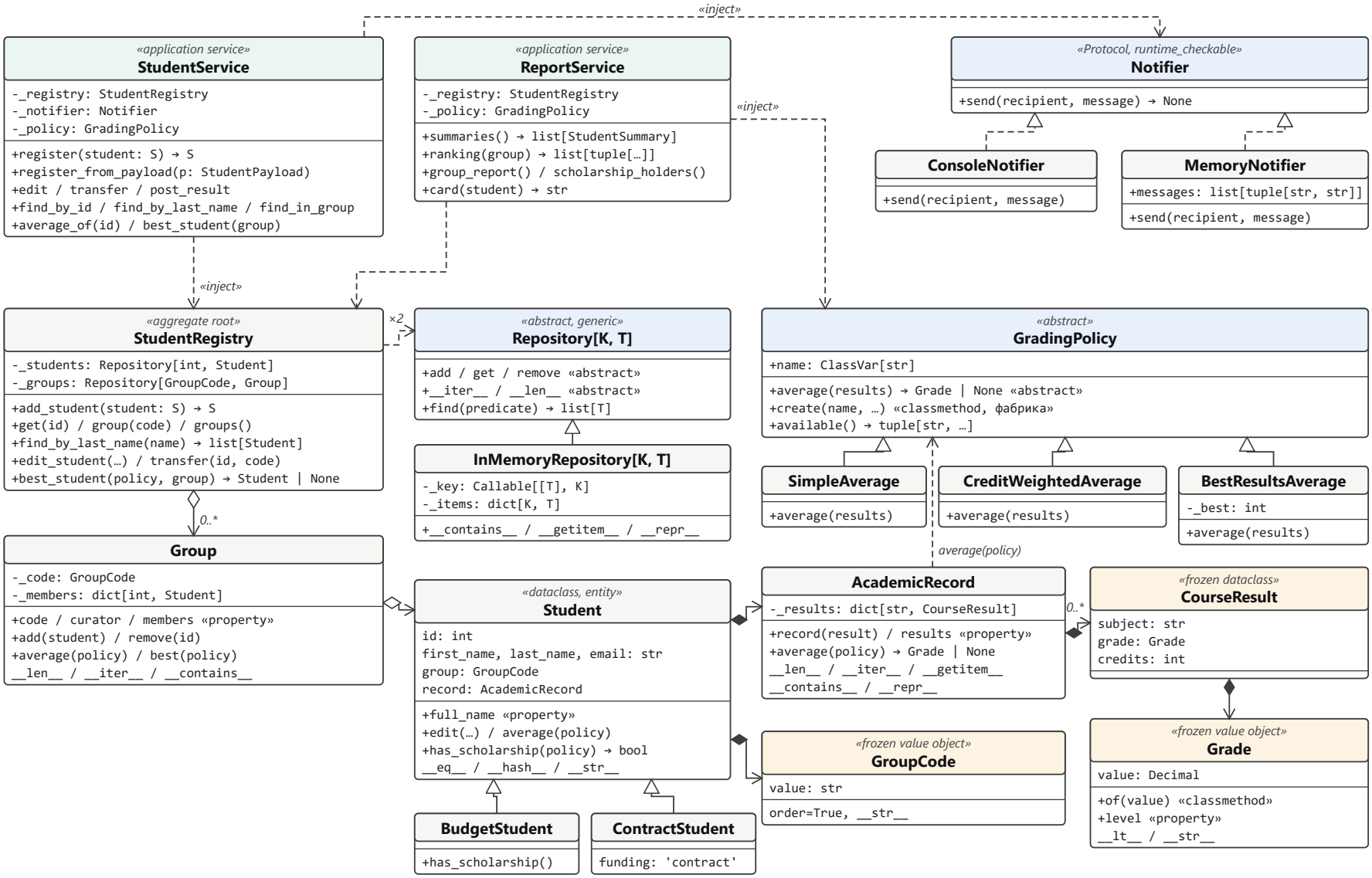


Рисунок 1 – UML class diagram моделі (◇ агрегація, ◆ композиція, △ наслідування, пунктир із △ – реалізація Protocol, пунктирна стрілка – використання або «inject»)

3. Inheritance, composition, encapsulation

Inheritance застосовано там, де є справжнє is-a. `GradingPolicy` – ABC з абстрактним `average`; кожен підклас оголошується з іменем (`class SimpleAverage(GradingPolicy, name="simple")`) і реєструється через `__init_subclass__`, тому фабрика `GradingPolicy.create(name)` знає всі стратегії без жодного списку. `BudgetStudent` і `ContractStudent` є студентами: базовий `has_scholarship` повертає False, а бюджетний підклас перевизначає його.

```
class GradingPolicy(ABC):
    name: ClassVar[str]
    _registry: ClassVar[dict[str, type["GradingPolicy"]]] = {}

    def __init_subclass__(cls, *, name: str, **kwargs: Any) -> None:
        super().__init_subclass__(**kwargs)
        cls.name = name
        GradingPolicy._registry[name] = cls

    @classmethod
    def create(cls, name: str, **options: Any) -> "GradingPolicy":
        try:
            policy_class = cls._registry[name]
        except KeyError:
            raise ValueError(f"Невідома політика оцінювання: {name!r}") from None
        return policy_class(**options)

    @abstractmethod
    def average(self, results: Iterable[CourseResult]) -> Grade | None: ...

    ...

class CreditWeightedAverage(GradingPolicy, name="weighted"):
    def average(self, results: Iterable[CourseResult]) -> Grade | None:
        items = list(results)
        credits = sum(result.credits for result in items)
        if not credits:
            return None
        points = sum((result.grade.value * result.credits for result in items), Decimal(0))
        return Grade(points / credits)

# class BudgetStudent(Student)
def has_scholarship(self, policy: GradingPolicy) -> bool:
    average = self.average(policy)
    return average is not None and average >= SCHOLARSHIP_MINIMUM
```

Composition (has-a): студент має `AcademicRecord` і `GroupCode`, запис має `CourseResult`, група має студентів, реєстр має два репозиторії, сервіс – реєстр, notifier і політику. **Encapsulation**: `full_name` – property без setter, `Group.members` і `AcademicRecord.results` повертають копію-кортеж, а студент потрапляє в групу лише через `Group.add`, який узгоджує поле `student.group`.

```
@property
def full_name(self) -> str:
    return f"{self.first_name} {self.last_name}"

@property
def members(self) -> tuple[Student, ...]:
    return tuple(self._members.values())

def add(self, student: Student) -> None:
    if student.id in self._members:
        raise DuplicateKeyError(f"Студент {student.id} уже є в групі {self._code}")
    self._members[student.id] = student
    student.group = self._code
```

4. Dataclass і immutable value object

`Grade` – frozen dataclass зі `slots=True`: незмінний, hashable, порівнюється за значенням. У `__post_init__` перевіряється діапазон 0–100 і значення нормалізується до сотих; `total_ordering` добудовує `<= > >=` з `__lt__`, `of()` свідомо не приймає float, а `level` – property з рівнем успішності. `Student` – змінюваний dataclass з `eq=False`: рівність і хеш визначає лише `id` (entity), а поля перевіряються в `__post_init__`.

```
@total_ordering
@dataclass(frozen=True, slots=True)
class Grade:
    value: Decimal

    def __post_init__(self) -> None:
        if not isinstance(self.value, Decimal):
            raise TypeError("value має бути Decimal; використовуйте Grade.of()")
        if not self.value.is_finite() or not 0 <= self.value <= MAX_POINTS:
            raise InvalidGradeError(f"Оцінка поза межами 0-100: {self.value}")
        object.__setattr__(self, "value", self.value.quantize(HUNDREDTH, ROUND_HALF_UP))

    @classmethod
    def of(cls, value: Decimal | int | str) -> Self:
        if isinstance(value, float):
            raise TypeError("float не підходить для оцінок: використовуйте str або Decimal")
        try:
            return cls(Decimal(value))
        except InvalidOperation:
            raise InvalidGradeError(f"Оцінка не є числом: {value!r}") from None

    @property
    def level(self) -> str:
        return next(label for threshold, label in LEVELS if self.value >= threshold)

    def __lt__(self, other: "Grade") -> bool:
        return self.value < other.value

    ...
```

```
@dataclass(eq=False, slots=True)
class Student:
    id: int
    first_name: str
    last_name: str
    email: str
    group: GroupCode
    record: AcademicRecord = field(default_factory=AcademicRecord)
    funding: ClassVar[str] = "none"

    def __post_init__(self) -> None:
        if self.id <= 0:
            raise InvalidStudentDataError("id студента має бути додатним")
        _check_person(self.first_name, self.last_name, self.email)

    def edit(
        self,
        *,
        first_name: str | None = None,
        last_name: str | None = None,
        email: str | None = None,
    ) -> None:
        new_first = self.first_name if first_name is None else first_name
        new_last = self.last_name if last_name is None else last_name
        new_email = self.email if email is None else email
        _check_person(new_first, new_last, new_email)
        self.first_name, self.last_name, self.email = new_first, new_last, new_email

    def __eq__(self, other: object) -> bool:
        return isinstance(other, Student) and other.id == self.id

    ...
```

5. ABC, Protocol, generics і TypedDict

`Repository` – ABC (nominal typing) і водночас generic class. `K` – constrained TypeVar (ключ лише `int` або `GroupCode`), `T` – звичайний TypeVar, `S` у реєстрі й сервісі – bound TypeVar: `register(BudgetStudent(...))` повертає саме `BudgetStudent`.

```
K = TypeVar("K", int, GroupCode)
T = TypeVar("T")
S = TypeVar("S", bound=Student)

class Repository(ABC, Generic[K, T]):
    @abstractmethod
    def add(self, item: T) -> None: ...

    @abstractmethod
    def get(self, key: K) -> T | None: ...

    @abstractmethod
    def remove(self, key: K) -> bool: ...

    @abstractmethod
    def __iter__(self) -> Iterator[T]: ...

    @abstractmethod
    def __len__(self) -> int: ...

    def find(self, predicate: Callable[[T], bool]) -> list[T]:
        return [item for item in self if predicate(item)]
```

`Notifier` – Protocol (structural typing): адаптери його не успадковують, але сумісні за сигнатурою `send`; завдяки `@runtime_checkable` працює `isinstance`. Три вузькі протоколи `Parser`, `Validator`, `Formatter` – interface segregation. TypedDict описує зовнішній JSON-payload студента; під час виконання це звичайний dict, тому його перетворює на доменний об'єкт `student_from_payload`.

```
@runtime_checkable
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None: ...

class StudentPayload(TypedDict):
    id: int
    first_name: str
    last_name: str
    email: str
    group: str
    kind: Literal["budget", "contract"]
    results: NotRequired[list[ResultPayload]]
```

Таблиця 3 – Використані dunder methods

Клас	Dunder methods	Призначення
`Grade`	`__lt__ __str__ __post_init__`	порядок (total_ordering), вивід, нормалізація
`Student`	`__eq__ __hash__ __str__ __post_init__`	тотожність за id, вивід, валідація
`AcademicRecord`	`__len__ __iter__ __getitem__ __contains__ __repr__`	custom collection: len, for, record['Математика'], in
`Group`	`__len__ __iter__ __contains__ __repr__`	`3 in group`, `for student in group`
`InMemoryRepository`	`__len__ __iter__ __contains__ __getitem__`	`1 in repo`, `repo[key]`
`GradingPolicy`	`__init_subclass__`	автоматична реєстрація підкласів у фабриці

6. Application service, dependency injection і SOLID

`StudentService` отримує реєстр, notifier і політику через конструктор і залежить лише від абстракцій; конкретні класи створюються в одному місці – composition root (`build_registry`, `build_services`). Один реєстр можна під'єднати до сервісів із різними notifier.

```
def __init__(
    self,
    registry: StudentRegistry,
    notifier: Notifier,
    policy: GradingPolicy,
) -> None:
    self._registry = registry
    self._notifier = notifier
    self._policy = policy

def register(self, student: S) -> S:
    self._registry.add_student(student)
    self._notify(student, f"вас зараховано до групи {student.group} ({student.funding})")
    return student

def build_registry() -> StudentRegistry:
    students = InMemoryRepository[int, Student](key=lambda student: student.id)
    groups = InMemoryRepository[GroupCode, Group](key=lambda group: group.code)
    return StudentRegistry(students, groups)

def build_services(
    registry: StudentRegistry, notifier: Notifier, policy: GradingPolicy
) -> tuple[StudentService, ReportService]:
    return StudentService(registry, notifier, policy), ReportService(registry, policy)
```

Таблиця 4 – Принципи SOLID у моделі

Принцип	Де реалізовано
SRP	`StudentRegistry` – інваріанти складу груп, `StudentService` – сценарії та сповіщення, `ReportService` – звіти, `GradingPolicy` – обчислення, `Grade` – валідація оцінки
OCP	нова стратегія – новий підклас з `name=...`: фабрика, реєстр, звіти не змінюються; новий канал сповіщень – новий клас із `send`
LSP	обидва види студентів виконують контракт `Student`; усі стратегії повертають `Grade | None` (None для порожнього запису) і взаємозамінні
ISP	`Parser`, `Validator`, `Formatter` – окремі протоколи; `Notifier` має один метод
DIP	сервіси залежать від `Notifier` (Protocol), `GradingPolicy` і `Repository` (ABC), а не від конкретних класів

7. Виконання програми

```
PS C:\Github\prof_python\Lab_04> python -m domain_app.main
ЛР4, варіант 1: система обліку студентів

Завантажено 10 студентів у 3 групах; повідомлень у MemoryNotifier: 13
isinstance(ConsoleNotifier(), Notifier) = True
Політики оцінювання (фабрика GradingPolicy.create): ('simple', 'weighted', 'best')

Пошук:
  за ID 3:  3 Ірина Бондар (PI-22, budget)
  за прізвищем 'мельник': ['Олег Мельник', 'Максим Мельник']
  у групі PI-22: ['Ірина Бондар', 'Марія Бойко', 'Софія Гнатюк']

Середній бал Анни Коваль за різними політиками (strategy):
  simple    -> 91.50
  weighted  -> 92.04
  best      -> 92.50
  найкращий студент (simple): Ірина Бондар; у групі PI-23: Катерина Лисенко
  найкращий студент (weighted): Ірина Бондар; у групі PI-23: Катерина Лисенко

Стипендія (поліморфний has_scholarship, поріг для BudgetStudent - 82):
  ['Анна Коваль', 'Ірина Бондар', 'Андрій Шевченко', 'Катерина Лисенко']
  види студентів: ['BudgetStudent', 'ContractStudent']; BudgetStudent - підклас Student: True

Редагування, переведення та перездача (повідомлення йдуть у ConsoleNotifier):
```

```
  [повідомлення -> oleh.melnyk@lnu.edu.ua] Олег Мельник: дані оновлено: Олег Мельник, oleh.melnyk@lnu.edu.ua
  [повідомлення -> student10@lnu.edu.ua] Максим Мельник: вас переведено до групи PI-22
  [повідомлення -> student8@lnu.edu.ua] Дмитро Ткаченко: Математика: 70.00, середній бал 63.90
  розміри груп після переведення: {'PI-21': 3, 'PI-22': 4, 'PI-23': 3}

Помилки предметної області:
  оцінка 105: InvalidGradeError: Оцінка поза межами 0-100: 105
  оцінка 'abc': InvalidGradeError: Оцінка не є числом: 'abc'
  невідомий студент 99: StudentNotFoundError: Студента 99 не знайдено
  дубль id 1: DuplicateKeyError: Студент із id 1 уже зареєстрований
  код групи PI-2: InvalidGroupCodeError: Некоректний код групи: 'PI-2'
  порожнє ім'я: InvalidStudentDataError: Ім'я та прізвище не можуть бути порожніми
  група PI-99: GroupNotFoundError: Групу PI-99 не знайдено

Звіти (ReportService):
  summaries -> TypedDict:
    {'id': 1, 'full_name': 'Анна Коваль', 'group': 'PI-21', 'kind': 'budget', 'average': '91.50', 'results': 3}
    {'id': 2, 'full_name': 'Олег Мельник', 'group': 'PI-21', 'kind': 'contract', 'average': '84.00', 'results':
3}
    {'id': 3, 'full_name': 'Ірина Бондар', 'group': 'PI-22', 'kind': 'budget', 'average': '95.20', 'results':
3}
  топ-4: Бондар 95.20, Лисенко 93.10, Коваль 91.50, Шевченко 88.70
  {'group': 'PI-21', 'students': 3, 'average': '88.07', 'best': 'Анна Коваль'}
  {'group': 'PI-22', 'students': 4, 'average': '77.98', 'best': 'Ірина Бондар'}
  {'group': 'PI-23', 'students': 3, 'average': '76.47', 'best': 'Катерина Лисенко'}
Картка: Ірина Бондар, PI-22, середній бал 95.20
  Математика: 94.00 (5 кр.)
  Програмування: 98.00 (6 кр.)
  Англійська: 93.60 (3 кр.)

Dunder methods:
  repr(record)=AcademicRecord(results=3), len=3, 'Математика' in record: True, record['Математика'] ->
Математика: 94.00 (5 кр.)
  Grade: max(...)=95.20, Grade.of('90') >= Grade.of('89.99'): True
  Group: Group(code='PI-22', students=4), 3 in group: True
  Student: == за id: True, hash збігається: True, рівень: відмінно

Inheritance vs composition (рядок із ЛР3):
  inheritance (CsvRowProcessor): 3: Ірина Бондар, PI-22, 95.20 (відмінно)
  composition CsvParser  + TextFormatter : 3: Ірина Бондар, PI-22, 95.20 (відмінно)
  composition CsvParser  + ShortFormatter: PI-22 #3 95.20
  composition JsonParser + TextFormatter : 12: Оксана Мороз, PI-24, 97.00 (відмінно)
  composition JsonParser + ShortFormatter: PI-24 #12 97.00
```

Рисунок 2 – Виконання демонстраційного сценарію

Перевірка вручну для Анни Коваль (90/5 кредитів, 95/6, 89.5/3): простий середній (90 + 95 + 89.5) / 3 = 91.50; зважений (90·5 + 95·6 + 89.5·3) / 14 = 92.04; середнє двох найкращих (95 + 90) / 2 = 92.50. Після переведення Максима до PI-22 середній бал групи PI-21 – (91.50 + 84.00 + 88.70) / 3 = 88.07, а Дмитро після перездачі математики на 70 має (70 + 62 + 59.7) / 3 = 63.90.

8. Статична перевірка типів (mypy)

Використано mypy зі `strict = true` (налаштування в `pyproject.toml`, `mypy_path = "src"`). Перевірка `src` і `tests` проходить без зауважень. Обов'язковий експеримент: у файл `experiments/type_errors.py` навмисно внесено три помилки з методички та чотири порушення доменного контракту:

```
PS C:\Github\prof_python\Lab_04> mypy experiments/type_errors.py
experiments\type_errors.py:6: error: Incompatible types in assignment (expression has type "str", variable has
type "int")  [assignment]
experiments\type_errors.py:10: error: Incompatible return value type (got "str", expected "int")  [return-
value]
experiments\type_errors.py:14: error: Argument 1 to "add" of "InMemoryRepository" has incompatible type
"Group"; expected "Student"  [arg-type]
experiments\type_errors.py:17: error: Property "full_name" defined in "Student" is read-only  [misc]
experiments\type_errors.py:18: error: Argument 1 to "of" of "Grade" has incompatible type "float"; expected
"Decimal | int | str"  [arg-type]
experiments\type_errors.py:19: error: Cannot instantiate abstract class "GradingPolicy" with abstract attribute
```

```
"average"  [abstract]
experiments\type_errors.py:20: error: Value of type variable "K" of "InMemoryRepository" cannot be "str"
[type-var]
Found 7 errors in 1 file (checked 1 source file)
```

Таблиця 5 – Знайдені помилки та виправлення (experiments/type_errors_fixed.py)

№	Помилка	Виправлення
1	`student_id: int = "1"`	`student_id: int = 1`
2	`def total() -> int: return "100"`	`return 100`
3	`Group` у `InMemoryRepository[int, Student]`	окремий `InMemoryRepository[GroupCode, Group]`
4	присвоєння `student.full_name` (property без setter)	`student.edit(first_name=...)`
5	`Grade.of(91.5)` – float	`Grade.of("91.5")`
6	екземпляр абстрактного `GradingPolicy`	`SimpleAverage()` з анотацією `GradingPolicy`
7	ключ `str` для constrained `K`	`InMemoryRepository[int, Student]`

```
PS C:\Github\prof_python\Lab_04> mypy experiments/type_errors_fixed.py experiments/di_comparison.py
Success: no issues found in 2 source files
```

```
PS C:\Github\prof_python\Lab_04> mypy   # src і tests, strict
Success: no issues found in 15 source files
```

9. Експериментальна частина

Dependency injection. Варіант A (`HardwiredStudentService`) сам створює реєстр і `ConsoleNotifier`; варіант B – `StudentService` з DI. Обидва виконують однакову реєстрацію студента:

```
PS C:\Github\prof_python\Lab_04> python experiments/di_comparison.py
A (hardwired): [повідомлення -> anna@example.com] Анна Коваль: вас зараховано
B (DI):        [('anna@example.com', 'Анна Коваль: вас зараховано до групи PI-21 (budget)')]
```

Таблиця 6 – Порівняння варіантів A і B

Критерій	A: залежність усередині	B: dependency injection
тестування	лише перехопленням stdout і розбором тексту	`MemoryNotifier`: повідомлення доступні як дані
заміна	зміна коду класу	інший об'єкт у конструкторі
DIP	порушено: залежить від конкретних класів	виконано: Protocol і ABC
coupling	2 конкретні класи всередині сервісу	0 – лише абстракції

Inheritance vs composition. Обробку рядка студента (parse → validate → format, формат ЛР3) реалізовано двома способами: `CsvRowProcessor` успадковує template method від `BaseRowProcessor`, а `RowProcessor(parser, validator, formatter)` складається з трьох компонентів-протоколів; обидва дають однаковий результат (перевірено тестом).

Таблиця 7 – Порівняння inheritance і composition

Критерій	Inheritance	Composition
coupling	підклас залежить від деталей template method базового класу	процесор знає лише три протоколи
extensibility	n форматів × m форматувань = n·m підкласів	n + m незалежних класів (2 parsers + 2 formatters = 4 комбінації)
testability	тестується лише весь процесор	кожен компонент тестується окремо, решту можна підмінити

Критерій	Inheritance	Composition
reuse	форматування прив'язане до ієрархії	`ShortFormatter` працює з будь-яким parser

10. Перевірка правильності

56 тестів (unittest) перевіряють value objects і їх межі, стратегії оцінювання на відомих даних, фабрику, custom collection, entity-рівність, поліморфну стипендію, групу, generic-репозиторій, реєстр (дублікати, переведення, найкращий), сервіси з `MemoryNotifier`, звіти, TypedDict, runtime_checkable Protocol, рівність результатів inheritance і composition та весь демонстраційний сценарій. Усі перевірки запускає CI-like команда:

```
PS C:\Github\prof_python\Lab_04> python tools/check.py
== ruff check
All checks passed!
== ruff format
15 files already formatted
== mypy --strict
Success: no issues found in 15 source files
== unittest
....................................................
----------------------------------------------------------------------
Ran 56 tests in 0.003s

OK
ALL CHECKS PASSED
```

Висновки

Побудовано типізовану об'єктну модель системи обліку студентів: entity `Student` з двома підкласами, група й академічний запис як колекції, value objects `Grade`, `GroupCode`, `CourseResult`, ABC-стратегії оцінювання з фабрикою, generic-репозиторій, Protocol для сповіщень і application services із dependency injection. Наслідування використано лише для справжнього is-a, решту поведінки зібрано композицією, що підтвердило порівняння: DI і композиція зменшують coupling і спрощують тестування. mypy у strict-режимі знайшов усі 7 навмисних помилок, а код проєкту й тестів проходить перевірку без зауважень.